

Rapport de Stage

Master 2 Informatique - Mathématique
Parcours Math-Info



Titre du Stage :

Étude sur les fonctions APN en dimension six

Réalisé par :

Abdoulaye LO

Encadrants :

Phillipe LANGEVIN, Valérie GILLOT

Année universitaire 2024–2025

Remerciements

Avant tout, je tiens à exprimer ma profonde gratitude à Monsieur Langevin pour ses explications claires et précieuses. Il n' a ménagé aucun effort pour m'accompagner dans ma compréhension, et son dévouement mérite toute ma reconnaissance.

J'adresse également mes sincères remerciements à Madame Gillot, qui prend même le soin de rédiger des notes pour me permettre de reprendre aisément le travail une fois de retour à la maison. Son attention et son soutien m'ont été d'une aide précieuse.

Je n'oublie pas non plus l'assistance apportée par ChatGPT, qui m'a été utile tant dans la rédaction de ce mémoire que dans la conception et la vérification des scripts informatiques.

À toutes et à tous, je vous remercie énormément.

Table des matières

Remerciements	1
Introduction	3
1 Présentation du laboratoire	4
1.1 Historique et activités	4
1.2 Organisation	4
1.3 Le rôle du stagiaire	5
1.4 Contexte et objectifs du stage	5
2 Fondements théoriques	6
2.1 Préliminaires	6
2.1.1 Fonctions booléennes et vectorielles	6
2.1.2 Forme normale algébrique et degré algébrique	6
2.1.3 Caractère – Transformée de Walsh – Uniformité différentielle	7
2.2 Fonctions APN	9
2.2.1 Définitions et propriétés	9
2.2.2 Fonctions à r niveaux spectraux	11
2.3 Fonctions courbes	12
2.4 Relation d'équivalence	13
2.5 Invariant	14
2.5.1 Invariant pour la CCZ-équivalence	14
2.5.2 Invariant pour la EA-équivalence	14
3 Méthodologie et mise en œuvre	15
4 Expérimentation et analyse des résultats	19
Conclusion	23
Bibliographie	24

Introduction

Les boîtes de substitution ou S-boxes jouent un rôle fondamental dans le domaine de la cryptographie symétrique, en particulier dans les algorithmes de chiffrement par blocs. Elles sont modélisées par des fonctions vectorielles conçues pour résister à des techniques de cryptanalyse telles que l'attaque différentielle et l'attaque linéaire.

Afin de garantir cette résistance, certaines classes de fonctions sont particulièrement recherchées, notamment les fonctions presque parfaitement non linéaires (APN), en raison de leurs propriétés cryptographiques remarquables.

Une des relations d'équivalence essentielles dans ce contexte est l'équivalence CCZ, qui permet de regrouper ces fonctions en classes partageant des structures similaires.

À ce jour, en dimension six, quatorze classes d'équivalence CCZ distinctes de fonctions APN ont été identifiées. La question de l'existence d'éventuelles classes supplémentaires demeure ouverte.

Par ailleurs, il a longtemps été conjecturé qu'il n'existait pas de permutations APN en dimension paire. Cette conjecture a toutefois été infirmée par Browning, Dillon, McQuistan et Wolfe [1], qui ont découvert une permutation APN en dimension six : la permutation de Dublin.

Notre travail s'inscrit dans le cadre du projet ab-APN, mené au sein du laboratoire IMATH, qui vise à explorer une méthode originale de construction de fonctions vectorielles APN en dimension six, en se basant sur l'analyse de la permutation de Dublin, pour construire de nouvelles classes de fonctions, potentiellement non équivalentes aux classes déjà connues.

Dans ce cadre, nous nous proposons dans un premier temps d'étudier en détail la structure des fonctions appartenant aux classes déjà identifiées, dans le but de mieux comprendre leurs propriétés. Dans un second temps, nous cherchons à classifier des fonctions dites courbes, dont le but est de construire de nouvelles fonctions APN que nous espérons appartenir à une classe CCZ encore inédite.

Après avoir présenté, le laboratoire d'accueil et le sujet de stage proposé, les notions fondamentales nécessaires à la compréhension du sujet sont introduites. Ensuite la problématique et la méthodologie qui ont permis de fournir les résultats numériques. Enfin, une analyse de ces résultats est menée afin d'en dégager les principaux enseignements.

Chapitre 1

Présentation du laboratoire

1.1 Historique et activités

L’Institut de Mathématiques de Toulon (IMATH) est une unité de recherche de l’Université de Toulon (UTLN), identifiée sous le numéro EA 2134. Il est issu, en 2006, de la fusion du Laboratoire d’Analyse Appliquée et Méthodes Numériques (ANAM) et du Groupe de Recherche en Informatique et Mathématiques (GRIM).

IMATH constitue une composante essentielle du paysage académique régional. Il s’illustre par son implication dans des collaborations scientifiques de portée nationale et internationale.

L’Institut rassemble 23 enseignants-chercheurs, un ingénieur de recherche, des chercheurs émérites, ainsi qu’une dizaine de doctorants et quatre stagiaires. Tous œuvrent dans des domaines variés relevant des mathématiques appliquées et de l’informatique théorique. Il a pour principales missions : la production de recherches scientifiques de qualité ; la formation par la recherche via l’encadrement de masters et de thèses ; le transfert de connaissances vers les secteurs industriels et scientifiques partenaires.

1.2 Organisation

L’IMATH est structuré autour de trois axes thématiques qui reflètent la diversité de ses domaines de recherche. Il se compose de trois équipes :

- Équipe d’Informatique et Algèbre Appliquée (IAA),
- Équipe d’Analyse Non Linéaire Appliquée (AA),
- Équipe de Modélisation Mathématique, Mécanique et Numérique (M3N),

couvrant des domaines allant de la cryptographie aux EDP non linéaires et à la simulation numérique de phénomènes physiques complexes.

Le laboratoire est dirigé par un directeur, épaulé par un conseil de laboratoire, et fonctionne en coordination avec les autres structures de recherche de l’université. Il dispose également de services de soutien administratif, informatique et logistique. L’IMATH participe actuellement au projet ANR SWAP financé par l’Agence Nationale de la Recherche qui contribue avec l’université de Toulon au financement de mon stage.

1.3 Le rôle du stagiaire

Durant mon stage au sein de l'IMATH de Toulon, j'ai été intégré à l'équipe IAA *Informatique et Algèbre Appliquée*.

Mes principales missions ont été les suivantes :

- Implanter des scripts en *Sage* ou en *Shell* pour effectuer le classement de fonctions selon différents critères d'équivalence ;
- Analyser les résultats et rédiger des notes ;
- Participer à des réunions hebdomadaires avec mes encadrants ;
- Assister au séminaire de l'équipe IAA favorisant l'échange scientifique et la compréhension des axes de recherche en cours.

Ce stage m'a permis de me familiariser avec le fonctionnement d'un laboratoire de recherche, de renforcer mes compétences scientifiques et techniques, et de mieux appréhender les exigences du travail en équipe dans un contexte académique.

1.4 Contexte et objectifs du stage

Dans le cadre de mon cursus de Master 2 en Informatique-Mathématique, option Math-Info, j'effectue un stage de quatre mois au sein du laboratoire IMATH de l'Université de Toulon. Ce stage s'inscrit dans un projet de recherche portant sur l'étude des fonctions booléennes cryptographiques, éléments fondamentaux dans la conception de primitives sécurisées en cryptographie symétrique.

L'objectif principal de ce stage était d'apporter une contribution à l'étude des fonctions vectorielles en dimension six, en lien avec la cryptographie et la théorie des fonctions APN. Plus précisément, il a été question de répondre à deux interrogations fondamentales :

- réaliser une **analyse fine des 14 classes d'équivalence CCZ** ;
- faire une **classification des fonctions (6,3)-courbes**.

Ces questions visent à mieux comprendre la structure et les propriétés de ces fonctions d'intérêt cryptographique. Elles sont au cœur du projet ab-APN.

Dans la partie implémentation, nous avons souhaité faire découvrir la bibliothèque **SboxU**, développée par Léo Perrin, qui s'appuie sur l'environnement **SageMath**. Cette bibliothèque offre un ensemble d'outils spécialement conçus pour manipuler et analyser des applications vectorielles, en particulier les *S-boxes* utilisées dans les algorithmes de chiffrement. Elle permet, de manière simple et efficace, de calculer des propriétés fondamentales telles que le degré algébrique, le spectre différentiel ou encore le spectre de Walsh, rendant ainsi plus accessible l'étude de ces objets complexes.

Chapitre 2

Fondements théoriques

2.1 Préliminaires

On considère le corps fini à deux éléments $\mathbb{F}_2 = \{0, 1\}$ et l'espace vectoriel $\mathbb{F}_2^m$ formé des vecteurs binaires de longueur m . On note par $q = 2^m$, le cardinal de $\mathbb{F}_2^m$.

2.1.1 Fonctions booléennes et vectorielles

Une **fonction booléenne** f en m variables est une application de $\mathbb{F}_2^m$ dans $\mathbb{F}_2$. Autrement dit, elle associe à chaque vecteur binaire de longueur m un bit.

Une **fonction vectorielle** F , on dit aussi une (m, n) -fonction, est une application de $\mathbb{F}_2^m$ dans $\mathbb{F}_2^n$. Elle est représentée par ses n fonctions booléennes coordonnées :

$$F(x) = (f_1(x), f_2(x), \dots, f_n(x)),$$

où $f_i = e_i \cdot F(x)$ avec e_i vecteur de la base canonique de $\mathbb{F}_2^n$.

On note l'ensemble des fonctions composantes de F par :

$$\text{comp}(F) = \{F_b \mid b \in \mathbb{F}_2^n\},$$

où chaque composante F_b est une fonction booléenne définie par :

$$F_b(x) = b \cdot F(x)$$

avec $\cdot$ le produit scalaire, $b \cdot F(x) = b_1 f_1(x) + b_2 f_2(x) + \dots + b_n f_n(x)$.

2.1.2 Forme normale algébrique et degré algébrique

Toute fonction booléenne $f : \mathbb{F}_2^m \rightarrow \mathbb{F}_2$ peut être exprimée de manière unique sous forme d'un polynôme dans l'anneau quotient :

$$\mathbb{F}_2[x_1, \dots, x_m] / (x_1^2 + x_1, \dots, x_m^2 + x_m).$$

Cette représentation est appelée **forme normale algébrique** (ou *Algebraic Normal Form*, ANF). Elle s'écrit :

$$\begin{aligned} ANF(f) &= \sum_{v \in \mathbb{F}_2^m} c_v x^v = f(x) \\ \sum_{(v_1, \dots, v_m) \in \mathbb{F}_2^m} c_v \prod_{i=1}^m x_i^{v_i} &= f(x_1, \dots, x_m) \end{aligned}$$

où chaque c_v , élément de $\mathbb{F}_2$, est un coefficient du polynôme.

Le **degré algébrique** d'une fonction booléenne, noté $\deg(f)$, est le maximum des degrés des monômes présents dans sa forme ANF.

Exemple. Pour la fonction booléenne suivante :

$$f(x_1, x_2) = x_1 + x_2 + x_1x_2,$$

le degré algébrique est 2, car le monôme x_1x_2 est de degré 2.

Dans le cas d'une fonction vectorielle $F = (f_1, \dots, f_n)$, on définit son degré algébrique par :

$$\deg(F) = \max_{1 \leq i \leq n} \deg(f_i).$$

Par construction, le degré algébrique d'une fonction définie sur $\mathbb{F}_2^m$ ne peut excéder m .

2.1.3 Caractère – Transformée de Walsh – Uniformité différentielle

Définition 1. Le *caractère* associé à un vecteur $c \in \mathbb{F}_2^m$ est la fonction :

$$\mu_c : \mathbb{F}_2^m \rightarrow \mathbb{Z}^*, \quad \mu_c(x) = (-1)^{c \cdot x}.$$

Remarque. Le produit scalaire étant bilinéaire, il en résulte que :

$$\mu_c(x + y) = \mu_c(x)\mu_c(y).$$

Propriété 1. Pour tout $x \in \mathbb{F}_2^m$, on a :

$$\sum_{c \in \mathbb{F}_2^m} \mu_c(x) = \begin{cases} q, & \text{si } x = 0; \\ 0, & \text{sinon.} \end{cases}$$

Preuve. Si $x = 0$, alors pour tout c , on a $\mu_c(0) = 1$, donc la somme vaut q .

Si $x \neq 0$, alors la fonction $c \mapsto c \cdot x$ est non constante, et prend autant de fois la valeur de 0 que la valeur de 1. Ainsi, la moitié des termes valent $+1$, l'autre -1 , donc la somme est nulle. $\square$

Définition 2. La *transformée de Walsh* d'une fonction booléenne en m variables f est définie par :

$$\forall a \in \mathbb{F}_2^m, \quad \hat{f}(a) = \sum_{x \in \mathbb{F}_2^m} (-1)^{f(x) + a \cdot x}.$$

Dans le cas d'une (m, n) -fonction F , pour $a \in \mathbb{F}_2^m$, $b \in \mathbb{F}_2^n$, on a introduit :

$$\hat{F}_b(a) = \sum_{x \in \mathbb{F}_2^m} (-1)^{b \cdot F(x) + a \cdot x} = \sum_{x \in \mathbb{F}_2^m} \mu_b(F(x))\mu_a(x).$$

Le moment spectral d'ordre 4 est donné par :

$$\kappa(f) = \frac{1}{q^2} \sum_{a \in \mathbb{F}_2^m} \hat{f}(a)^4,$$

et pour une fonction vectorielle F , on pose :

$$Q(F) = \# \{ (x, y, z, t) \in (\mathbb{F}_2^m)^4 \mid x + y + z + t = 0, F(x) + F(y) + F(z) + F(t) = 0 \}.$$

Proposition 1. Pour une (m, n) -fonction F , on a :

$$Q(F) = \sum_{b \in \mathbb{F}_2^n} \kappa(F_b) = \frac{1}{2^n q} \sum_{b \in \mathbb{F}_2^n} \sum_{a \in \mathbb{F}_2^m} \widehat{F}_b(a)^4.$$

Preuve. Pour tout $(x, y, z, t) \in (\mathbb{F}_2^m)^4$ et par la méthode des sommes de caractères, on a :

$$\begin{aligned} Q(F) &= \sum_{x, y, z, t} \frac{1}{2^n} \sum_{b \in \mathbb{F}_2^n} \mu_b(F(x) + F(y) + F(z) + F(t)) \frac{1}{q} \sum_{a \in \mathbb{F}_2^m} \mu_a(x + y + z + t) \\ &= \frac{1}{2^n q} \sum_{b \in \mathbb{F}_2^n} \sum_{a \in \mathbb{F}_2^m} \sum_{x, y, z, t} \mu_b(F(x) + F(y) + F(z) + F(t)) \mu_a(x + y + z + t) \\ &= \frac{1}{2^n q} \sum_{b \in \mathbb{F}_2^n} \sum_{a \in \mathbb{F}_2^m} \left(\sum_{x \in \mathbb{F}_2^m} \mu_b(F(x)) \mu_a(x) \right)^4 \\ &= \frac{1}{2^n q} \sum_{b \in \mathbb{F}_2^n} \sum_{a \in \mathbb{F}_2^m} \widehat{F}_b(a)^4. \end{aligned}$$

□

Soit F une (m, n) -fonction. Pour tout $u \in \mathbb{F}_2^m \setminus \{0\}$ et $v \in \mathbb{F}_2^n$, on définit :

$$N(u, v) = \# \{ (x, y) \in \mathbb{F}_2^m \times \mathbb{F}_2^m \mid x + y = u, F(x) + F(y) = v \}.$$

C'est aussi :

$$N(u, v) = \# \{ x \in \mathbb{F}_2^m \mid F(x) + F(x + u) = v \}.$$

Ce nombre est toujours pair, car si (x, y) est une solution de $x + y = u$ et $F(x) + F(y) = v$, alors (y, x) en est également une.

Propriété 2. Pour tout $u \in \mathbb{F}_2^m \setminus \{0\}$ et $v \in \mathbb{F}_2^n$, on a :

$$N(u, v) = \frac{1}{2^n q} \sum_{b \in \mathbb{F}_2^n} \sum_{a \in \mathbb{F}_2^m} \widehat{F}_b(a)^2 \mu_b(v) \mu_a(u).$$

Preuve. On a, pour tout $(x, y) \in (\mathbb{F}_2^m)^2$

$$\begin{aligned} N(u, v) &= \frac{1}{2^n q} \sum_{x, y} \sum_{b \in \mathbb{F}_2^n} \sum_{a \in \mathbb{F}_2^m} \mu_b(F(x) + F(y) + v) \mu_a(x + y + u) \\ &= \frac{1}{2^n q} \sum_{b \in \mathbb{F}_2^n} \sum_{a \in \mathbb{F}_2^m} \left(\sum_{x \in \mathbb{F}_2^m} \mu_b(F(x)) \mu_a(x) \right) \left(\sum_{y \in \mathbb{F}_2^m} \mu_b(F(y)) \mu_a(y) \right) \mu_b(v) \mu_a(u) \\ &= \frac{1}{2^n q} \sum_{b \in \mathbb{F}_2^n} \sum_{a \in \mathbb{F}_2^m} \widehat{F}_b(a)^2 \mu_b(v) \mu_a(u). \end{aligned}$$

□

Définition 3. L'*uniformité différentielle* d'une (m, n) -fonction F , notée $\delta(F)$, est définie par :

$$\delta(F) = \max \{ N(u, v) : u \in \mathbb{F}_2^m \setminus \{0\}, v \in \mathbb{F}_2^n \}$$

L'uniformité différentielle mesure la résistance d'une fonction à la cryptanalyse différentielle. Plus elle est faible, meilleure est la résistance.

2.2 Fonctions APN

En cryptographie symétrique, les fonctions ayant une faible uniformité différentielle sont particulièrement recherchées dans la conception de chiffrements par bloc. C'est notamment le cas des (m, m) -fonctions dites *presque parfaitement non linéaires*, ou *APN* (pour *Almost Perfect Nonlinear* en anglais).

2.2.1 Définitions et propriétés

Définition 4. Une (m, m) -fonction F est dite APN si son uniformité différentielle est égale à deux.

Lemme 1. Une (m, m) -fonction F est APN si, et seulement si pour x, y, z, t éléments distincts de $\mathbb{F}_2^m$ tels que $x + y + z + t = 0$, alors

$$F(x) + F(y) + F(z) + F(t) \neq 0.$$

Preuve. Soit F une (m, m) -fonction, si $(x, y, z, t) \in (\mathbb{F}_2^m)^4$ est une solution du système

$$\begin{cases} x + y + z + t = 0, \\ F(x) + F(y) + F(z) + F(t) = 0, \end{cases} \quad (2.1)$$

alors, en posant $u = x + y = z + t$ et $v = F(x) + F(y) = F(z) + F(t)$, on peut réécrire le système sous la forme :

$$\begin{cases} x + y = u = z + t, \\ F(x) + F(y) = v = F(z) + F(t). \end{cases}$$

Ainsi, (x, y) , (y, x) , (z, t) et (t, z) et sont des solutions du système :

$$\begin{cases} x + y = u, \\ F(x) + F(y) = v. \end{cases} \quad (2.2)$$

Dans le cas où la fonction F est APN, pour tout $u \neq 0$, le système d'équation (2.2) admet deux solutions.

Soit (x, y) et (y, x) ces solutions, on a bien : $0 \neq u = x + y = z + t$ et $F(z) + F(t) \neq v$.

Donc $x \neq y \neq z \neq t$ et $F(x) + F(y) \neq F(z) + F(t)$. $\square$

Proposition 2. Une (m, m) -fonction F est APN si, et seulement si,

$$\sum_{b \in \mathbb{F}_2^m \setminus \{0\}} \kappa(F_b) = 2(q - 1)q. \quad (2.3)$$

Preuve. Soit F (m, m) -fonction, par la proposition 1, on a :

$$Q(F) = \sum_{b \in \mathbb{F}_2^m} \kappa(F_b).$$

$Q(F)$ est aussi le nombre de solutions du système d'équation (2.1).

Si la fonction F est APN, alors d'après lemme 1, les solutions de (2.1) sont triviales (c'est-à-dire que les solutions ne sont pas tous distinctes).

- $x = y$ et $z = t$ avec q^2 possibilités,
- $x = z$ et $y = t$ avec q^2 possibilités,
- $x = t$ et $y = t$ avec q^2 possibilités,
- $x = y = z = t$ avec q possibilités, ce cas est compté 3 fois.

Il vient alors que $Q(F) = q^2 + q^2 + q^2 - 2q = 3q^2 - 2q$.

En égalant les deux expressions de $Q(F)$:

$$\sum_{b \in \mathbb{F}_2^m} \kappa(F_b) = 3q^2 - 2q.$$

Puisque $\kappa(F_0) = q^2$, on obtient :

$$\sum_{b \in \mathbb{F}_2^m} \kappa(F_b) = 3q^2 - 2q - q^2 = 2(q - 1)q.$$

□

Ainsi, pour construire des fonctions APN, il suffit de rechercher des fonctions vectorielles F vérifiant l'équation (2.3).

On peut commencer par s'intéresser aux (m, m) -fonction F qui satisfont :

$$\forall b \in \mathbb{F}_2^m \setminus \{0\}, \quad \kappa(F_b) = 2q.$$

De telles fonctions n'existent que dans le cas où m est impair.

Lemme 2. Si m est pair, il n'existe pas de fonction booléenne f telles que $\kappa(f) = 2q$.

Preuve. Soit f une fonction booléenne en m variables.

Par définition, le moment spectral d'ordre 4 est donné par :

$$\kappa(f) = \frac{1}{q^2} \sum_{a \in \mathbb{F}_2^m} \hat{f}(a)^4.$$

$\forall a \in \mathbb{F}_2^m, \hat{f}(a)^3 \equiv \hat{f}(a) \pmod{3}$. Il en résulte : $\hat{f}(a)^4 = \hat{f}(a) \cdot \hat{f}(a)^3 \equiv \hat{f}(a)^2 \pmod{3}$.

Donc,

$$\sum_{a \in \mathbb{F}_2^m} \hat{f}(a)^4 \equiv \sum_{a \in \mathbb{F}_2^m} \hat{f}(a)^2 \pmod{3}.$$

Or, par la relation de Parseval, on sait que :

$$\sum_{a \in \mathbb{F}_2^m} \hat{f}(a)^2 = q^2.$$

On en déduit alors :

$$\kappa(f) = \frac{1}{q^2} \sum_{a \in \mathbb{F}_2^m} \hat{f}(a)^4 \equiv \frac{q^2}{q^2} = 1 \pmod{3}.$$

Par ailleurs, si m est pair, $q = 2^m \equiv 1 \pmod{3}$ alors $2q = 2^{m+1} \equiv 2 \pmod{3}$.

Donc si m est pair $\kappa(f) \neq 2q$.

□

Il n'existe donc pas de fonctions vectorielles en dimension paire pour lesquelles κ prend une seule valeur. Nous nous intéressons dès lors au cas des fonctions pour lesquelles κ prend exactement deux valeurs distinctes. Ce sont les fonctions dites à deux niveaux spectraux.

2.2.2 Fonctions à r niveaux spectraux

Définition 5. On dira qu'une (m, m) -fonction F est à r niveaux spectraux (distincts) $(\alpha_1, \dots, \alpha_r)$ si :

$$\forall b \in \mathbb{F}_2^m \setminus \{0\}, \quad \frac{1}{q}\kappa(F_b) \in \{\alpha_1, \dots, \alpha_r\} \quad \text{et} \quad \frac{1}{q}\alpha(F_0) = q.$$

Cette définition permet de classer les (m, m) -fonction selon leurs nombre de niveaux spectraux.

Notre étude s'appuie sur cette notion en examinant la permutation dite de Dublin, qui se distingue par le fait qu'elle présente précisément deux niveaux spectraux, à savoir $\alpha_1 = 1.75$ pour 56 composantes et $\alpha_2 = 4$ pour 7 composantes.

Par ailleurs, Calderini dans son article[2] a déjà déterminer le nombre de classes EA contenues dans chaque classe CCZ. En appliquant la définition des fonctions à r niveaux spectraux, nous pouvons approfondir ces résultats, en offrant une répartition plus détaillée des classes CCZ selon leurs caractéristiques spectraux.

0	54	48	13	15	18	53	35
25	63	45	52	3	20	41	33
59	36	2	34	10	8	57	37
60	19	42	14	50	26	58	24
39	27	21	17	16	29	1	62
47	40	51	56	7	43	44	38
31	11	4	28	61	46	5	49
9	6	23	32	30	12	55	22

TABLE 1 – Permutation de Dublin

Cette représentation correspond à la table de vérité de la permutation présentée lors de la conférence[3] par Dillon.

Elle est obtenue en calculant, pour chaque vecteur x de $\mathbb{F}_2^m$, l'image $F(x)$, puis en convertissant cette image en un entier en base décimale.

Les lignes de la table sont ordonnées selon l'ordre croissant des entiers correspondant aux vecteurs x . Par exemple 25 correspond à $F(8)$.

2.3 Fonctions courbes

Les fonctions *courbes*, ou hautement non linéaires, jouent un rôle important dans la conception de primitives cryptographiques, en raison de leur forte résistance aux attaques linéaires. Elles ont été introduites et classifiées pour la dimension six par Rothaus en 1976 (voir l'article[8]). La classification en dimension 8 a été annoncée lors de la conférence BFA-2024 par Gillot ; Langevin et Polujan.

Définition 6. La *distance de Hamming* entre deux fonctions booléennes en m variables f, g est définie par :

$$d_H(f, g) = \# \{x \in \mathbb{F}_2^m \mid f(x) \neq g(x)\}.$$

Définition 7. La *non-linéarité* d'une fonction booléenne en m variables f est donnée par :

$$nl(f) = \min_{deg(l) \leq 1} d_H(f, l).$$

Pour une (m, n) -fonction F , sa non-linéarité est définie à partir de la non-linéarité de ses fonctions composantes :

$$nl(F) = \min_{b \in \mathbb{F}_2^n \setminus \{0\}} nl(F_b).$$

Définition 8. Une fonction booléenne à m variables f est dite *courbe*, si :

$$\forall a \in \mathbb{F}_2^m, \hat{f}(a) = \pm 2^{\frac{m}{2}}.$$

Définition 9. Une (m, n) -fonction F est dite *courbe* si :

$$\forall b \in \mathbb{F}_2^n \setminus \{0\}, F_b \in \text{comp}(F), \forall a \in \mathbb{F}_2^m \hat{F}_b(a) = \pm 2^{\frac{m}{2}}.$$

Autrement dit les $2^n - 1$ fonctions composantes sont courbes.

On dira que F est (m, n) -courbe

Il a été montré par Nyberg que les fonctions courbes existent uniquement lorsque m est pair et $n \leq m/2$ (voir l'article[6]).

Les fonctions courbes peuvent également être utilisées pour construire des fonctions APN. Cette idée conduit à la notion suivante :

Définition 10 (Extension APN). Soient F une (m, m) -fonction et G une (m, n) -courbe.

On dit que F est une extension APN de G si :

- $F = (g_1, \dots, g_n, f_{n+1}, \dots, f_m)$,
- et F est une fonction APN.

2.4 Relation d'équivalence

Les relations d'équivalence des (m, n) -fonctions qui préservent l'uniformité différentielle et la non-linéarité permettent de réduire le nombre de fonctions à étudier et à classer, trois relations d'équivalence ont été définies :

Définition 11. Soient F, G deux (m, n) -fonctions.

- *Équivalence affine* : on dit que F et G sont affinement équivalentes s'il existe deux permutations affines $A : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ et $B : \mathbb{F}_2^m \rightarrow \mathbb{F}_2^m$ telles que :

$$G = A \circ F \circ B.$$

- *Équivalence affine étendue (EA)* : les fonctions F et G sont dites EA-équivalentes s'il existe deux permutations affines $A : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$, $B : \mathbb{F}_2^m \rightarrow \mathbb{F}_2^m$ et une fonction affine $C : \mathbb{F}_2^m \rightarrow \mathbb{F}_2^n$, telles que :

$$G = A \circ F \circ B + C.$$

- *Équivalence CCZ (Carlet–Charpin–Zinoviev)* : on dit que F et G sont CCZ-équivalentes s'il existe une permutation affine $L : \mathbb{F}_2^m \times \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m \times \mathbb{F}_2^n$ telle que :

$$L(\Gamma(F)) = \Gamma(G),$$

où $\Gamma(F) = \{(x, F(x)) \mid x \in \mathbb{F}_2^m\}$ est le graphe de F .

En notant par F_Γ et G_Γ les fonctions indicatrices respectives de $\Gamma(F)$ et $\Gamma(G)$, c'est-à-dire les fonctions de $\mathbb{F}_2^{m+n}$ dans $\mathbb{F}_2$ définies par :

$$F_\Gamma(x, y) = \begin{cases} 1 & \text{si } (x, y) \in \Gamma(F), \\ 0 & \text{sinon,} \end{cases}$$

alors F_Γ et G_Γ sont EA-équivalentes.

Remarque. L'EA-équivalence est un cas particulier de la CCZ-équivalence.

Exemple Si une (m, n) -fonction F est une permutation alors F et F^{-1} sont CCZ-équivalentes mais pas EA-équivalentes.

En effet,

$$\Gamma(F) = \{(x, F(x)) \mid x \in \mathbb{F}_2^m\} = \{(F^{-1}(t), t) \mid t \in \mathbb{F}_2^m\}.$$

Par la transformation $L : (x, y) \rightarrow (y, x)$, on a :

$$L(\Gamma(F)) = L(\{(F^{-1}(t), t) \mid t \in \mathbb{F}_2^m\}) = \{(t, F^{-1}(t)) \mid t \in \mathbb{F}_2^m\} = \Gamma(F^{-1}).$$

Nous avons vérifié par calcul que la permutation de Dublin et son inverse ne sont pas EA-équivalentes.

2.5 Invariant

La notion d'*invariants* jouent un rôle important dans le processus de classification des fonctions. Il s'agit de propriétés mathématiques conservées au sein d'une classe d'équivalence. Ils permettent de distinguer efficacement certaines classes sans avoir à effectuer de tests d'équivalence exhaustifs, qui sont généralement coûteux.

Définition 12. Soit G un groupe, X un ensemble, et une application

$$j : X \longrightarrow \mathbb{Z}.$$

On dit que j est un **invariant** de l'action de G sur X si :

$$\forall \phi \in G, \forall x \in X, \quad j(\phi \cdot x) = j(x).$$

Autrement dit, j est constant sur chaque orbite de l'action de G sur X .

Rappel : $\forall x \in X, \text{Orb}(x) := \{\phi \cdot x \mid \phi \in G\}$.

On considère la relation d'équivalence définie par l'action :

$$\forall x, y \in X, \quad x \sim y \iff \exists \phi \in G \text{ tel que } \phi \cdot x = y, \text{ c'est-à-dire } y \in \text{Orb}(x).$$

Proposition 3. Pour tous $x, y \in X$, si $x \sim y$, alors $j(x) = j(y)$.

On utilise souvent l'invariant via la contraposée de la proposition pour montrer que deux objets ne sont pas équivalentes.

Dans cette section, nous présentons seulement deux invariants utilisés pour distinguer les classes d'équivalence, notamment sous les relations de type CCZ et EA. Pour une étude plus approfondie et une liste étendue d'invariants, nous renvoyons le lecteur à l'article de Kaleski[5].

2.5.1 Invariant pour la CCZ-équivalence

Définition 13. Le *spectre de Walsh étendu* d'une (m, n) -fonction F est l'ensemble suivant :

$$\mathfrak{w} = \left\{ \left\{ |\widehat{F}_b(a)| \mid a, b \in \mathbb{F}_2^m \right\} \right\}.$$

Le spectre de Walsh étendu est invariant par équivalence CCZ. Autrement dit, si deux fonctions sont CCZ-équivalentes, elles possèdent le même spectre de Walsh étendu.

2.5.2 Invariant pour la EA-équivalence

Soit F une (m, n) -fonction, et soit $\langle F \rangle$ l'espace vectoriel sur $\mathbb{F}_2$ engendré par les n fonctions booléennes coordonnées de F .

Pour toute fonction $f \in \langle F \rangle$, on pose :

$$\mathcal{J}(F) := \{ \{ \mathfrak{w}(f + g) \mid g \in \mathcal{Q} \} \}$$

où $\mathcal{Q}$ est l'ensemble des formes quadratiques de rang 2.

Définition 14. Pour une (m, n) -fonction F , le multi-ensemble suivant :

$$\mathcal{J}'(F) := \{ \{ \mathcal{J}(f) \mid f \in \langle F \rangle \} \}$$

définit un invariant EA.

L'invariant $\mathcal{J}'$, présenté dans l'article [4], a été utilisé comme outil de pré-classification dans nos expérimentations, en amont de l'identification des classes d'équivalence EA.

Chapitre 3

Méthodologie et mise en œuvre

Le projet de construction ab-APN a pour objectif de construire des fonctions APN en dimension 6 dont les fonctions booléennes composantes non nulles appartiennent à seulement deux classes affines. Pour une telle fonction, l'ensemble des $\kappa(f)$, où f parcourt les composantes, est réduit à une paire $\{a, b\}$.

ab-pairs¹ : $aX + bY = 2(q - 1), X + Y = q - 1, a = \kappa(f), b = \kappa(f)$.

En notant $K(c, F)$ le sous-espace engendré par les composantes f telles que $\kappa(f) = c$, le défi consistant à construire toutes les fonctions APN F de type (1.75, 4.00) telles que $\deg(F) = 4$ et $K(4, F)$ soit un espace de fonctions cubiques a été relevé avec succès.

Ensuite, la question de la résolution des problèmes suivants s'est posée.

Problème 1

Pour les 14 classes d'équivalence CCZ, déterminer :

- pour chaque classe, le nombre de sous-classes d'équivalence EA ;
- le nombre de *niveaux spectraux* distincts des fonctions qu'elles contiennent.

Problème 2

Classifier 2132 fonctions (6, 3)-courbes en classes d'équivalence CCZ.

Méthodologie

1. Nous avons pris des représentants des 14 classes CCZ.
 - Pour chaque fonction F , nous avons utilisé la fonction `enumerate_ea_classes(F)` de la bibliothèque `sboxU`, laquelle renvoie une liste contenant au moins un représentant de chaque classe d'équivalence EA incluse dans la classe d'équivalence CCZ de F .
 - Une pré-classification a été effectuée à l'aide de l'invariant $\mathcal{J}$.
 - Une classification EA complète a ensuite été réalisée sur les fonctions obtenues.
 - Le nombre de niveaux spectraux a été déterminé pour chaque fonction.

1. Nouveau concept : fonction à deux niveaux spectraux (ab-pairs)

2. Étapes pour classifier les 2132 fonctions (6,3)-courbes :

- Utilisation d'invariant CCZ pour un pré-classification,
- Réécriture de la fonction `are_ccz_equivalent(F,G)` de la bibliothèque `sboxU`, initialement conçue pour tester l'équivalence CCZ entre (m,m) -fonctions, afin de l'appliquer au cas des (m,n) -fonctions avec $n < m$.

Le tableau suivant donne, pour chaque représentant des 14 CCZ-classes, le nombre classes EA après classification.

F	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10	F11	F12	F13	F14
<code>enum(F)</code>	25	94	118	94	190	222	118	190	126	106	106	118	106	114
# classes EA	25	19	91	19	3	13	91	3	92	85	85	91	13	86

Les temps de calcul (en seconde) indiqués sont approximatifs et susceptibles d'être supérieurs en pratique.

2 (6,6)-fonctions	CCZ	EA
$\sim$	0.60 – 1.01	1 – 90
$\not\sim$	1 – 208	1.13 – 335

2 (6,3)-courbes	CCZ
$\sim$	0.84 – 270
$\not\sim$	2.72 – 4500

Implantation

Concernant la partie implantation, plusieurs scripts ont été développés :

- un script permettant de calculer les niveaux spectraux d'une fonction vectorielle ;
- un script dédié à la classification selon les équivalences CCZ et EA, que nous présentons par la suite ;
- un script permettant de vérifier si les fonctions composantes ayant les mêmes niveaux spectraux forment un sous-espace vectoriel.
- un script `bash` permettant de répartir les tâches sur plusieurs machines.

Dans nos calculs, nous utilisons la table de vérité des fonctions étudiées. Cette table énumère, pour chaque entrée binaire possible, la valeur de sortie correspondante, codée sous forme entière.

```

import argparse
import os
import sys
sys.path.append('/home/lo/Bureau/stage/sboxU')
import time
from sboxU import*

#---Generator matrices of the linear codes associated with f---
def graph(f):
    n = int(log(len(f), 2))
    m = max(f).bit_length()
    l1 = [[1] + tobin((x << m) | f[x], n + m) for x in range(len(f))]
    l2 = [[0] + [0]*n + tobin(y, m) for y in range(1, 2**m)]
    C1 = Matrix(GF(2), len(f), 1 + n + m, l1)
    C2 = Matrix(GF(2), len(l1) + len(l2), 1 + n + m, l1 + l2)
    return C1, C2

#---Test for CCZ-equivalence---
def are_ccz_equiv(f, g):
    if len(f) != len(g):
        return False
    code_f = LinearCode(graph(f)[0].transpose())
    code_g = LinearCode(graph(g)[0].transpose())
    return code_f.is_permutation_equivalent(code_g)

#---Test for EA-equivalence---
def are_ea_equiv(f, g):
    if len(f) != len(g):
        return False
    code_f = LinearCode(graph(f)[1].transpose())
    code_g = LinearCode(graph(g)[1].transpose())
    return code_f.is_permutation_equivalent(code_g)

#---CCZ classification for a list of functions---
def classify_ccz(sboxes):
    cl= [sboxes[0]]
    for sbox in sboxes[1:]:
        if all(not are_ccz_equiv(sbox, member) for member in cl):
            cl.append(sbox)
    return cl

#---EA classification for a list of functions---
def classify_ea(sboxes):
    cl= [sboxes[0]]
    for sbox in sboxes[1:]:
        if all(not are_ea_equiv(sbox, member) for member in cl):
            cl.append(sbox)
    return cl

```

Listing 3.1 – Script Sage pour la classification CCZ et EA (Partie 1)

```

#---Read file containing the functions "TT= "
def read_functions(fichier):
    functions = []
    with open(fichier, "r") as f:
        for ligne in f:
            if ligne.startswith("TT="):
                valeurs = list(map(int, ligne.strip().split("=")[1].split()))
                functions.append(valeurs)
    return functions

#---Save the results after classification---
def save_classes(cl, name_base_file):
    exit_name = f"rep_{name_base_file}.txt"
    with open(exit_name, 'w') as f:
        for i, classe in enumerate(cl, start=1):
            line = "TT= " + " ".join(str(x) for x in classe)
            f.write(line + "\n")
    print(f"{len(cl)} representatives saved in : {exit_name}")

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Classifiy EA or CCZ.")
    parser.add_argument("chemin", help="Chemin vers un fichier contenant les TT")
    parser.add_argument("--mode", choices=["ea", "ccz"], default="ea",
                        help="Type de classification (ea ou ccz). Par default : ea.")
    args = parser.parse_args()

    fonctions = read_functions(args.chemin)
    base_name = os.path.splitext(os.path.basename(args.chemin))[0]

    if args.mode == "ea":
        classes = classify_ea(fonctions)
    else:
        classes = classify_ccz(fonctions)

    save_classes(classes, base_name + "_" + args.mode)

#--- For execution : sage script.py file.txt --mode ea (ou --mode ccz)---

```

Listing 3.2 – Script Sage pour la classification CCZ et EA (Partie 2)

Pour implanter ce script, nous avons exploité l'équivalence entre les fonctions et leurs codes linéaires associés, telle que décrite dans le document[7], page 32.

Une (n, m) -fonction F est associée aux matrices suivantes :

$$C_1(F) := \begin{pmatrix} 1 \\ \mathbf{x} \\ F(\mathbf{x}) \end{pmatrix}, \quad C_2(F) := \begin{pmatrix} 1 & 0 \\ \mathbf{x} & \mathbf{0} \\ F(\mathbf{x}) & \mathbf{y} \end{pmatrix}, \quad \mathbf{x} \in \mathbb{F}_2^n, \mathbf{y} \in \mathbb{F}_2^m \setminus \{\mathbf{0}\}.$$

Ces matrices constituent les génératrices des codes linéaires associés à la fonction F .

Équivalences

1. Deux fonctions F et G sont *CCZ-équivalentes* si et seulement si les codes linéaires $C_1(F)$ et $C_1(G)$ sont équivalents.
2. Deux fonctions F et G sont *EA-équivalentes* si et seulement si les codes linéaires $C_2(F)$ et $C_2(G)$ sont équivalents.

Chapitre 4

Expérimentation et analyse des résultats

1. Analyse des 14 CCZ-classes

Après les calculs, nous avons retrouvé la **classe CCZ** de la permutation de Dublin, représentée par la fonction F6 (6^e fonction dans 14 CCZ classes).

Dans cette classe, nous avons identifié **treize classes EA** distinctes.

Parmi celles-ci :

- trois correspondent à des fonctions à *deux niveaux spectraux* ;
- cinq à des fonctions à *trois niveaux* ;
- et les cinq restantes à des fonctions à *quatre niveaux*.

Le *nombre de classes EA des composantes* est indiqué entre crochets dans le tableau, et la couleur verte signale que ces composantes forment un *sous-espace vectoriel* de $\mathbb{F}_2^6$.

Les **deux classes EA** contenant chacune une **permutation** figurent sur la *deuxième ligne*(en gras) de la Table 6.

Enfin, **toutes les quatorze classes** (y compris celle de la permutation) contiennent un représentant à *deux niveaux spectraux*, **à l'exception de deux d'entre elles F1 et F4**. Ces représentants présentent des *caractéristiques structurelles identiques*.

2. CCZ classification des (6, 3)-courbes

Treize classes d'équivalence CCZ ont été identifiées, avec des représentants de **degré algébrique deux** ou **trois**. La forme ANF de ces représentants est donnée dans la Table 15.

Les résultats des tables suivantes constituent un complément au tableau 1 de l'article de Calderini [2], en précisant, pour chaque classe CCZ, le nombre de niveaux spectraux des fonctions qu'elle contient.

TABLE 1 – Répartition des Comp(F1)

# EA classes	Degré			$\frac{1}{q}\kappa(F_b)$				
	2	3	4	1.0	1.75	2.5	3.25	4.0
2	3	60		25[3]		34[3]		4[3]
2	1	62		22[2]		40[3]		1[1]
2	1	62		22[3]		40[5]		1[1]
2	1	62		24[2]		36[3]		3[2]
1	1	62		23[2]		38[3]		2[2]
1	31	32		28[2]		28[1]		7[1]
1	1	30	32	18[2]	8[1]	36[5]		1[1]
1	1	30	32	18[2]	8[2]	36[5]		1[1]
5	1	30	32	18[2]	8[2]	36[4]		1[1]
2	1	30	32	20[2]	8[2]	32[3]		3[2]
1	1	30	32	20[2]	8[2]	32[4]		3[2]
1	1	30	32	20[2]	8[2]	32[5]		3[2]
1	1	30	32	20[2]	8[1]	32[5]		3[2]
1		15	48	8[2]	28[5]	26[5]		1[1]
1		15	48	8[3]	32[4]	18[6]	4[1]	1[1]
1		15	48	8[3]	32[3]	18[6]	4[1]	1[1]
Total : 25								

TABLE 3 – Répartition des Comp(F3)

# EA classes	Degré			$\frac{1}{q}\kappa(F_b)$				
	2	3	4	1.0	1.75	2.5	3.25	4.0
1	63			42[1]				21[1]
3	1	62		22[2]		40[2]		1[1]
21	-	-		26[2]		32[2]		5[3]
9	-	-		26[2]		32[2]		5[2]
27	-	-		30[2]		24[2]		9[3]
3	-	-		30[2]		24[2]		9[2]
3		31	32	8[1]	32[2]	20[3]		3[1]
1		-	-	8[1]	32[3]	20[2]		3[1]
3		-	-	8[2]	32[3]	20[2]		3[1]
1		-	-	8[2]	32[2]	20[2]		3[1]
1		-	-	8[1]	32[2]	20[2]		3[1]
1		-	-	12[1]	28[3]	16[3]	4[1]	3[1]
3		-	-	12[2]	28[3]	16[2]	4[1]	3[1]
2		-	-	12[1]	28[2]	16[2]	4[1]	3[1]
6		-	-	12[2]	28[2]	16[2]	4[1]	3[1]
1		-	-	12[1]	28[3]	16[2]	4[1]	3[1]
1		-	-	12[2]	28[3]	16[3]	4[1]	3[1]
1		-	-	12[1]	28[2]	16[3]	4[1]	3[1]
1		-	-	16[2]	24[3]	12[3]	8[2]	3[1]
2		-	-	16[2]	24[3]	12[2]	8[2]	3[1]
Total : 91								

TABLE 7 – Répartition des Comp(F7)

# EA classes	Degré			$\frac{1}{q}\kappa(F_b)$				
	2	3	4	1.0	1.75	2.5	3.25	4.0
1	63			42[1]				21[1]
3		63		24[2]		36[3]		3[1]
15	1	62		26[2]		32[2]		5[2]
24	-	-		26[2]		32[2]		5[3]
3	-	-		30[2]		24[2]		9[2]
18	-	-		30[2]		24[2]		9[3]
3	-	-		34[2]		16[2]		13[3]
1		31	32	8[1]	32[2]	20[3]		3[1]
2		-	-	8[1]	32[3]	20[3]		3[1]
1		-	-	8[1]	32[3]	20[2]		3[1]
1		-	-	8[2]	32[3]	20[2]		3[1]
1		-	-	8[2]	32[2]	20[2]		3[1]
5		-	-	12[2]	28[3]	16[3]	4[1]	3[1]
3		-	-	12[2]	28[3]	16[2]	4[1]	3[1]
5		-	-	12[2]	28[2]	16[2]	4[1]	3[1]
1		-	-	12[1]	28[2]	16[2]	4[1]	3[1]
1		-	-	12[2]	28[2]	16[1]	4[1]	3[1]
1		-	-	12[1]	28[3]	16[2]	4[1]	3[1]
1		-	-	12[2]	28[2]	16[3]	4[1]	3[1]
1		-	-	12[1]	28[2]	16[3]	4[1]	3[1]
Total : 91								

TABLE 2 – Répartition des Comp(F2)

# EA classes	Degré			$\frac{1}{q}\kappa(F_b)$				
	2	3	4	1.0	1.75	2.5	3.25	4.0
1	63			42[1]				21[1]
6	1	62		30[2]		24[2]		9[3]
3	1	62		26[2]		32[2]		5[3]
3	1	62		26[2]		32[2]		5[2]
3	1	62		22[2]		40[2]		1[1]
1		31	32	12[2]	28[3]	16[2]	4[1]	3[1]
1		31	32	12[2]	28[2]	16[2]	4[1]	3[1]
1		31	32	12[1]	28[2]	16[2]	4[1]	3[1]
Total : 19								

TABLE 4 – Répartition des Comp(F4)

# EA classes	Degré			$\frac{1}{q}\kappa(F_b)$						
	2	3	4	1.0	1.75	2.5	4.0	4.75	8.5	16.0
3	7	56		30[2]		32[1]				1[1]
1	63			46[1]			16[1]			1[1]
6	1	62		34[2]		22[2]	5[2]		2[1]	
3	1	62		34[2]		22[2]	5[3]		2[1]	
3	1	62		30[2]		30[2]	1[1]		2[1]	
2		31	32	16[2]	28[3]	12[2]	3[1]	4[1]		
1		31	32	16[2]	28[3]	12[3]	3[1]	4[1]		
Total : 19										

TABLE 5 – Répartition des Comp(F5)

# EA classes	Degré			$\frac{1}{q}\kappa(F_b)$				
	2	3	4	1.0	1.75	2.5	3.25	4.0
1	63			42[1]				21[1]
1	1	62		30[2]		24[2]		9[3]
1		31	32	16[2]	24[2]	12[2]	8[1]	3[1]
Total : 3								

TABLE 6 – Répartition des Comp(F6)

# EA classes	Degré			$\frac{1}{q}\kappa(F_b)$			
	2	3	4	1	1.75	2.5	4.0
1	63			42[1]			21[1]
2		7	56		56[1]		7[1]
5	1	62		30[2]		24[2]	9[3]
2		31	32	12[2]	32[3]	12[2]	7[2]
1		31	32	12[1]	32[2]	12[2]	7[2]
2		31	32	12[2]	32[2]	12[2]	7[2]
Total : 13							

TABLE 8 – Répartition des Comp(F8)

# EA classes	Degré			$\frac{1}{q}\kappa(F_b)$			
	2	3	4	1.0	1.75	2.5	4.0
1	63			42[1]			21[1]
1	1	62		30[2]		24[2]	9[3]
1		31	32	12[1]	32[2]	12[1]	7[2]
Total : 3							

TABLE 9 – Répartition des Comp(F9)

# EA classes	Degré			$\frac{1}{q}\kappa(F_b)$				
	2	3	4	1.0	1.75	2.5	3.25	4.0
1	63			42[1]				21[1]
1		63		24[2]		36[2]		3[1]
2			63	24[2]		36[3]		3[1]
15	1	62		26[2]		32[2]		5[2]
9	-	-		26[2]		32[2]		5[3]
3			63	28[2]		28[3]		7[2]
33	1	62		30[2]		24[2]		9[3]
3	-	-		34[2]		16[1]		13[3]
3	-	-		34[2]		16[2]		13[3]
1		31	32	12[1]	32[2]	12[2]		7[2]
1		-	-	12[1]	32[2]	12[3]		7[2]
1		-	-	12[1]	32[3]	12[3]		7[2]
3		-	-	12[2]	32[3]	12[2]		7[2]
1		15	48	8[1]	40[2]	8[1]		7[2]
1		31	32	8[1]	32[2]	20[3]		3[1]
1		-	-	8[1]	32[3]	20[3]		3[1]
1		-	-	8[2]	32[3]	20[3]		3[1]
5		-	-	12[2]	28[2]	16[2]	4[1]	3[1]
1		-	-	12[2]	28[3]	16[3]	4[1]	3[1]
3		-	-	16[2]	24[1]	12[2]	8[1]	3[1]
1		-	-	16[2]	24[2]	12[3]	8[1]	3[1]
2		-	-	16[2]	24[3]	12[2]	8[2]	3[1]
Total : 92								

TABLE 10 – Répartition des Comp(F10)

# EA classes	Degré			$\frac{1}{q}\kappa(F_b)$				
	2	3	4	1.0	1.75	2.5	3.25	4.0
1	63			42[1]				21[1]
3		63		24[2]		36[3]		3[1]
24	1	62		26[2]		32[2]		5[2]
15	-	-		26[2]		32[2]		5[3]
3	-	-		30[2]		24[2]		9[2]
18	-	-		30[2]		24[2]		9[3]
3	-	-		34[2]		16[2]		13[3]
1		31	32	8[1]	32[2]	20[3]		3[1]
2		-	-	8[1]	32[3]	20[2]		3[1]
3		-	-	8[2]	32[2]	20[2]		3[1]
2		-	-	12[1]	28[2]	16[2]	4[1]	3[1]
1		-	-	12[1]	28[2]	16[3]	4[1]	3[1]
1		-	-	12[1]	28[3]	16[3]	4[1]	3[1]
3		-	-	12[2]	28[2]	16[2]	4[1]	3[1]
2		-	-	12[2]	28[3]	16[2]	4[1]	3[1]
1		-	-	20[2]	20[2]	8[2]	12[2]	3[1]
2		-	-	20[2]	20[3]	8[2]	12[3]	3[1]
Total : 85								

TABLE 11 – Répartition des Comp(F11)

# EA classes	Degré			$\frac{1}{q}\kappa(F_b)$				
	2	3	4	1.0	1.75	2.5	3.25	4.0
1	63			42[1]				21[1]
3		63		24[2]		36[3]		3[1]
6	1	62		26[2]		32[2]		5[2]
33	1	62		26[2]		32[2]		5[3]
6	1	62		30[2]		24[2]		9[2]
15	1	62		30[2]		24[2]		9[3]
3	1	62		34[2]		16[2]		13[3]
3		31	32	8[1]	32[2]	20[2]		3[1]
2		31	32	8[1]	32[3]	20[2]		3[1]
1		31	32	8[2]	32[2]	20[2]		3[1]
2		31	32	12[1]	28[2]	16[3]	4[1]	3[1]
1		31	32	12[1]	28[3]	16[3]	4[1]	3[1]
4		31	32	12[2]	28[2]	16[2]	4[1]	3[1]
1		31	32	12[2]	28[2]	16[3]	4[1]	3[1]
2		31	32	12[2]	28[3]	16[2]	4[1]	3[1]
2		31	32	12[2]	28[3]	16[3]	4[1]	3[1]
Total : 85								

TABLE 12 – Répartition des Comp(F12)

# EA classes	Degré			$\frac{1}{q}\kappa(F_b)$				
	2	3	4	1.0	1.75	2.5	3.25	4.0
1	63			42[1]				21[1]
3		63		24[2]		36[3]		3[1]
18	1	62		26[2]		32[2]		5[3]
21	-	-		26[2]		32[2]		5[2]
3	-	-		30[2]		24[2]		9[2]
18	-	-		30[2]		24[2]		9[3]
3	-	-		34[2]		16[2]		13[3]
1		31	32	8[1]	32[2]	20[3]		3[1]
1		-	-	8[1]	32[3]	20[2]		3[1]
1		-	-	8[1]	32[3]	20[3]		3[1]
3		-	-	8[2]	32[3]	20[3]		3[1]
1		-	-	12[1]	28[2]	16[2]	4[1]	3[1]
2		-	-	12[1]	28[2]	16[3]	4[1]	3[1]
2		-	-	12[2]	28[2]	16[2]	4[1]	3[1]
7		-	-	12[2]	28[3]	16[2]	4[1]	3[1]
6		-	-	12[2]	28[3]	16[3]	4[1]	3[1]
Total : 91								

TABLE 14 – Répartition des Comp(F14)

# EA classes	Degré			$\frac{1}{q}\kappa(F_b)$				
	2	3	4	1.0	1.75	2.5	3.25	4.0
1	63			42[1]				21[1]
3		63		24[2]		36[3]		3[1]
15	1	62		26[2]		32[2]		5[2]
9	-	-		26[2]		32[2]		5[3]
3			63	28[2]		28[3]		7[2]
6	1	62		30[2]		24[2]		9[2]
27	-	-		30[2]		24[2]		9[3]
3	-	-		34[2]		16[1]		13[3]
3	-	-		34[2]		16[2]		13[3]
3		31	32	8[2]	32[2]	20[2]		3[1]
1		15	48	8[1]	40[3]	8[2]		7[2]
1		31	32	12[1]	28[3]	16[2]	4[1]	3[1]
2		-	-	12[2]	28[3]	16[2]	4[1]	3[1]
1		-	-	16[2]	28[3]	8[1]	4[1]	7[2]
5		-	-	16[2]	28[3]	8[2]	4[1]	7[2]
2		-	-	20[2]	20[2]	8[2]	12[2]	3[1]
1		-	-	20[2]	20[3]	8[2]	12[2]	3[1]
Total : 86								

TABLE 13 – Répartition des Comp(F13)

# EA classes	Degré			$\frac{1}{q}\kappa(F_b)$				
	2	3	4	1.0	1.75	2.5	3.25	4.0
1	63			42[1]				21[1]
6	1	62		30[2]		24[2]		9[3]
3	1	62		30[2]		24[2]		9[2]
1		31	32	16[2]	24[3]	12[3]	8[2]	3[1]
1		31	32	16[2]	24[3]	12[2]	8[2]	3[1]
1		31	32	16[2]	24[2]	12[2]	8[1]	3[1]
Total : 13								

Composante	Expression ANF
f_1^1	$ae + bd + be + cd + cf$
f_2^1	$ad + bd + bf + ce$
f_3^1	$af + be + cd$
f_1^2	$ab + ae + bd + be + cd + cf$
f_2^2	$ad + bd + bf + ce$
f_3^2	$af + be + cd$
f_1^3	$ab + ad + cf + de$
f_2^3	$ad + bd + bf + ce$
f_3^3	$af + be + cd$
f_1^4	$abc + ae + bd + be + cd + cf$
f_2^4	$ad + bd + bf + ce$
f_3^4	$af + be + cd$
f_1^5	$abd + ab + ade + ad + bcd + ce + de + df$
f_2^5	$abe + ac + bc + bf + de$
f_3^5	$abc + af + be + cd$
f_1^6	$abc + abf + ace + ad + ae + bce + bc + bd + be + cd + cef + df$
f_2^6	$abe + ac + bc + bf + de$
f_3^6	$abc + af + be + cd$
f_1^7	$abc + abf + ae + bc + be + cd + df$
f_2^7	$abe + ac + bc + bf + de$
f_3^7	$abc + af + be + cd$
f_1^8	$abd + abe + abf + ace + acf + ae + be + bf + cd$
f_2^8	$abe + abf + ace + ad + be + cf$
f_3^8	$abe + acd + af + bd + cd + ce$
f_1^9	$abd + abe + abf + ab + ace + acf + ac + adf + ae + be + bf + cd + def$
f_2^9	$abe + abf + ace + ad + be + cf$
f_3^9	$abe + acd + af + bd + cd + ce$
f_1^{10}	$abd + abf + acd + ad + ae + af + bcd + bce + bcf + be + bf + cd + ce$
f_2^{10}	$abe + abf + acd + acf + ae + bce + bd + be + bf + cd$
f_3^{10}	$abf + ace + ad + bcd + bd + be + cd + ce + cf$
f_1^{11}	$abd + abf + ab + acd + ad + ae + af + bcd + bce + bcf + be + bf + cd + ce$
f_2^{11}	$abe + abf + acd + acf + ae + bce + bd + be + bf + cd$
f_3^{11}	$abf + ace + ad + bcd + bd + be + cd + ce + cf$
f_1^{12}	$adf + aef + ae + af + bde + bdf + bef + cdf + cd + ce + ef$
f_2^{12}	$abe + abf + acd + acf + ae + bce + bd + be + bf + cd$
f_3^{12}	$abf + ace + ad + bcd + bd + be + cd + ce + cf$
f_1^{13}	$abd + abf + acd + ad + ae + af + bcd + bce + bcf + be + bf + cd + ce + def + de + df + ef$
f_2^{13}	$abe + abf + acd + acf + ae + bce + bd + be + bf + cd$
f_3^{13}	$abf + ace + ad + bcd + bd + be + cd + ce + cf$

TABLE 15 – Formes normales algébriques (ANF) des 13 fonctions (6,3)-courbes

Synthèse et conclusion

Big APN problem : « Existe-t-il une permutation APN en dimension paire strictement supérieure à 6 ? » est le problème fondamental qui guide le projet de construction ab-APN.

L’observation fine de la permutation de Dublin a mis en évidence une propriété remarquable : petit nombre de niveaux spectraux.

Dans le cadre de ce projet, il m’a été demandé de réaliser deux tâches numériques au moyen de la `sboxU`.

La première mission consistait à classifier les 14 classes CCZ selon le critère du nombre de niveaux spectraux. Cette analyse a permis, pour les fonctions à deux niveaux spectraux, d’identifier 12 représentants de classes CCZ de types similaires, ainsi que deux représentants de classes EA appartenant à une même classe CCZ. De plus, nous avons déterminé le nombre de niveaux spectraux pour tous les représentants des 14 classes CCZ.

La seconde étape portait sur la classification CCZ de 2132 fonctions $(6, 3)$ -courbes. Cette étude a abouti à la distinction de 13 classes différentes.

L’un des principaux défis rencontrés au cours de ce travail a été la durée parfois très longue des calculs, notamment lors de la classification des fonctions. Toutefois, cette contrainte m’a offert l’opportunité d’apprendre à concevoir des scripts shell permettant d’exploiter efficacement plusieurs machines en parallèle.

Ce stage a été une expérience enrichissante, tant sur le plan académique que personnel. Il m’a permis de mettre en pratique les connaissances acquises au cours de mon cursus de Master en informatique-mathématique, en particulier dans le domaine de l’algorithmique, de l’analyse mathématique des fonctions booléennes et vectorielles.

Bibliographie

- [1] K. A. BROWNING et al. « An APN permutation in dimension six ». In : *Finite Fields : Theory and Applications*. Sous la dir. de G. L. MULLEN et D. PANARIO. T. 518. Contemporary Mathematics. American Mathematical Society, 2010, p. 33-42. URL : <https://www.ams.org/bookstore/pspdf/conm-518-toc.pdf>.
- [2] Marco CALDERINI. « On the EA-classes of known APN functions in small dimensions ». In : *Cryptography and Communications* 12.4 (2020), p. 821-840. DOI : 10.1007/s12095-020-00427-1.
- [3] J. F. DILLON. « APN Polynomials : An Update ». In : *Proceedings of the 9th International Conference on Finite Fields and Their Applications (Fq9)*. Presentation slides available at <https://maths.ucd.ie/~gmg/Fq9Talks/Dillon.pdf>. Dublin, Ireland, 2009.
- [4] Valérie GILLOT, Philippe LANGEVIN et Abdoulaye LO. « On the Dublin permutation ». In : @<https://boolean.w.uib.no/bfa-2025/>. BFA 2024, Selmer Center. Larnaca, Cyprus, submitted 2025.
- [5] Nikolay S. KALEYSKI. « Invariants for EA- and CCZ-equivalence of APN and AB functions ». In : *Cryptography and Communications* 14.6 (2022), p. 1283-1305. DOI : 10.1007/s12095-021-00541-8.
- [6] Kaisa NYBERG. « Differentially uniform mappings for cryptography ». In : *Advances in Cryptology - EUROCRYPT '93*. T. 765. Lecture Notes in Computer Science. Springer-Verlag, 1994, p. 55-64. DOI : 10.1007/3-540-48285-7_6.
- [7] Alexandr POLUJAN. « Boolean and Vectorial Functions : A Design-Theoretic Point of View ». Eingereicht am : 14.04.2021, Verteidigung am : 05.07.2021. Gutachter : Prof. Dr. Alexander Pott, Prof. Dr. Enes Pasalic. Dissertation zur Erlangung des akademischen Grades doctor rerum naturalium (Dr. rer. nat.) Magdeburg, Germany : Otto-von-Guericke-Universität Magdeburg, juill. 2021.
- [8] Oscar S. ROTHBAUS. « On “bent” functions ». In : *Journal of Combinatorial Theory, Series A* 20.3 (1976), p. 300-305. DOI : 10.1016/0097-3165(76)90011-0.